

Company: Google

Interview Type: TECHNICAL

Q1: Root-to-Leaf Path Equalization

Difficulty: Hard

Tags: Binary Tree, Dynamic Programming, Greedy

Problem Statement: Given a binary tree with non-negative integer node values, find the minimum total increment (increasing node values by 1) required so that every path from the root to a leaf has the same sum.

Expected Approach: Use a bottom-up recursion to calculate the maximum path sum for each subtree. For a node to satisfy the condition, its left and right subtrees must eventually result in the same path sum.

Optimized Approach: Traverse the tree post-order. For each node, calculate the max path sum of its left and right children. The difference between these two sums is the amount that must be added to the "lighter" child's root to balance the subtrees greedily at the lowest possible level. Return the sum of increments and the new max path sum to the parent.

Time Complexity: $O(N)$ where N is the number of nodes.

Space Complexity: $O(H)$ where H is the height of the tree for recursion stack.

Optimizations: Avoid global variables by returning a pair/struct containing (current_max_path, total_increments).

Hints: Think greedily—where is the most efficient place to add a value to balance two paths? Does adding a value to a node affect all paths passing through it?

Q2: Shortest Path in a Toggling Binary Grid

Difficulty: Hard

Tags: Graph, BFS, State-Space Search

Problem Statement: You are given an $M \times N$ grid of 0s (passable) and 1s (walls). You have a remote that toggles the entire grid: all 0s become 1s and all 1s become 0s. You can press the button at most K times. Find the shortest path from $(0,0)$ to $(M-1, N-1)$. You can only move to a cell if it is a 0 in the current grid state. Pressing the button does not change your current position but counts as a "state change" that can be done before any move.

Expected Approach: Model the problem as a BFS on a state-space graph where each state is defined by $(row, col, toggles_used, current_grid_state)$.

Optimized Approach: Use a 3D BFS where the state is $(row, col, toggles_used)$. Since the grid state is binary (original or flipped), the $current_grid_state$ is simply $toggles_used \% 2$. Maintain a $visited[row][col][toggles_used]$ array to prevent cycles. In each step, you can either move to an

adjacent cell (if it's 0 in the current state) or stay in the same cell and press the toggle button (if `toggles_used < K`).

Time Complexity: $O(M * N * K)$

Space Complexity: $O(M * N * K)$

Optimizations: If the target cell $(M-1, N-1)$ is reached, return immediately. Use a bitmask if K is small, but for general K , a 3D array is optimal.

Hints: Each cell can be visited in two different "phases." How does the parity of K affect the accessibility of a cell (r, c) ? Is pressing the button twice in a row ever optimal?

Q3: Subarray Median Ranking

Difficulty: Hard

Tags: Binary Search, Fenwick Tree, Prefix Sums

Problem Statement: Given an array of N integers and a target value K , count the total number of subarrays whose median is greater than or equal to K . For a subarray of length m , the median is defined as the element at position $\lfloor (m+1)/2 \rfloor$ in the sorted version of the subarray.

Expected Approach: A brute force $O(N^3)$ or $O(N^2 \log N)$ approach involves checking every subarray. Transforming the problem: treat elements $\geq K$ as $+1$ and elements $< K$ as -1 . The median is $\geq K$ if the number of elements $\geq K$ is greater than the number of elements $< K$. This simplifies to finding the number of subarrays with a sum ≥ 0 .

Optimized Approach: 1. Transform the array into B where $B[i] = 1$ if $A[i] \geq K$ else -1 . 2. Compute prefix sums P of B . 3. The condition $\text{sum}(i, j) \geq 0$ becomes $P[j] - P[i-1] \geq 0$ or $P[j] \geq P[i-1]$. 4. Use a Fenwick Tree (Binary Indexed Tree) to count pairs (i, j) such that $i < j$ and $P[i] \leq P[j]$ in $O(N \log N)$.

Time Complexity: $O(N \log N)$

Space Complexity: $O(N)$

Optimizations: Coordinate compression on prefix sums if the range of values is large (though here it is bounded by $[-N, N]$).

Hints: How does the condition "median $\geq K$ " relate to the count of elements greater than K vs. less than K ? Can you use prefix sums to count ranges with positive sums?

Q4: K-Split Tree Maximization

Difficulty: Hard

Tags: Tree DP, Binary Search on Answer, Greedy

Problem Statement: Given a tree with N nodes where each node has a non-negative weight, you are allowed to remove exactly M edges to partition the tree into $M+1$ connected components. Find the maximum possible value of W , where W is the minimum weight among all resulting components (the weight of a component is the sum of weights of its nodes).

Expected Approach: Use binary search on the result W . For a fixed W , determine if it is possible to cut the tree into at least $M+1$ components such that each component's weight is $\geq W$.

Optimized Approach: For the fixed-value check, perform a bottom-up DFS. For each node, calculate the weight of the subtree rooted at it. If a subtree's total weight reaches $\geq W$, "cut" the edge connecting it to its parent and increment the component count. The remaining weight of the node is passed up to the parent only if it hasn't been cut. This greedy approach works because cutting as early as possible (at the lowest possible nodes) leaves more weight for the upper parts of the tree.

Time Complexity: $O(N \log (\sum \text{weights}))$

Space Complexity: $O(N)$

Optimizations: Prune binary search range using the total sum of weights divided by $M+1$ as the upper bound.

Hints: If you knew the minimum weight W , could you verify if it's achievable greedily from the leaves up? What happens to the "leftover" weight after a cut?

Q5: Robot Navigation with Battery Constraint and Charging Stations

Difficulty: Hard

Tags: Graph, Dijkstra, State-Space BFS

Problem Statement: You are given a weighted undirected graph representing a city map where weights are energy costs. A robot starts at node S with a battery capacity C . There are charging stations at specific nodes where the battery can be refilled to C for a fixed cost P_i . Find the path to node T that minimizes the total cost (sum of energy consumed + sum of charging costs), provided the robot never runs out of energy between stations.

Expected Approach: Use Dijkstra's algorithm where the state is defined as $(\text{current_node}, \text{current_battery})$. The state space size is $N * C$, which is inefficient if C is large.

Optimized Approach: Build a new graph where nodes are only the start, the target, and charging stations. For every pair of these nodes, run Dijkstra to find the shortest energy distance. If the distance $d \leq C$, add an edge in the new graph with weight $d + P_{\text{target}}$. Then, run Dijkstra on this reduced graph (size $K+2$ where K is the number of stations) to find the minimum cost to reach T .

Time Complexity: $O(K * (E + V \log V) + K^2)$, where K is the number of charging stations.

Space Complexity: $O(V + K^2)$ to store the reduced graph and Dijkstra states.

Optimizations: Use a priority queue for the initial Dijkstra runs and prune searches that exceed the battery capacity C .

Hints: Think about the charging stations as the only points where "decisions" about cost are made. How can you simplify the graph to only involve these points?

Q6: Maximum XOR Sum of Two Disjoint Subtrees

Difficulty: Hard

Tags: Tree, Trie, DFS Order, Dynamic Programming

Problem Statement: Given a tree of N nodes where each node has an integer value, find two disjoint subtrees such that the sum of their internal XOR values is maximized. (The XOR value of a subtree is the bitwise XOR of all node values within it).

Expected Approach: Calculate the XOR sum for every subtree using a post-order traversal. For each node, the "disjoint" part is tricky; you need to compare a subtree with everything "outside" it.

Optimized Approach: 1. Calculate $\text{subtree_xor}[u]$ for all u . 2. Use DFS order (entry/exit times) to flatten the tree. 3. For every node u , any subtree disjoint from it is either in a different branch or above it. 4. A more robust way: Use the property that if two subtrees are disjoint, one cannot be an ancestor of the other, OR they are in different branches of a common ancestor. 5. For every node, calculate the max subtree XOR in its subtree and the max subtree XOR outside it. Use a Trie to store all subtree_xor values encountered so far to find the best pair in $O(N \cdot \text{bit_length})$.

Time Complexity: $O(N \cdot \log(\max_val))$

Space Complexity: $O(N \cdot \log(\max_val))$ to store the Trie and tree arrays.

Optimizations: Use a single Trie and perform two passes (one for prefix max, one for suffix max) on the flattened DFS array to find the best disjoint XOR sum.

Hints: If you pick a node u , any node v that is not in the subtree of u and u is not in the subtree of v could potentially root a disjoint subtree. How can DFS order help identify these?

Q7: Longest Increasing Path in a Binary Tree with K Descendant Jumps

Difficulty: Hard

Tags: Dynamic Programming, Tree, Recursion

Problem Statement: Given a binary tree where each node has an integer value, find the length of the longest path where node values are strictly increasing. In this path, you are allowed to "jump" from a node to any of its descendants (not just immediate children) up to K times. A jump counts as one move between two nodes in the path.

Expected Approach: Use a naive DFS to explore all possible increasing paths from every node, using a recursion that tracks the current node and the number of jumps remaining.

Optimized Approach: Use memoization with DFS. For each node u and remaining jumps k , store the maximum length of an increasing path starting at u . To handle descendant jumps efficiently, each node should return a sorted list or a balanced BST of its children's reachable values to the parent, or perform a DFS where the state includes $(\text{node}, \text{jumps_left}, \text{last_val})$. More optimally, use a Tree DP where each node returns a data structure (like a Segment Tree or Fenwick Tree over the coordinate-compressed values of the tree) representing the max path lengths for each possible "last value" in its subtree.

Time Complexity: $O(N \cdot K \cdot \log N)$ with coordinate compression and Fenwick Tree.

Space Complexity: $O(N \cdot K)$ for memoization/storage.

Optimizations: Use coordinate compression on all tree values to use a Fenwick tree of size N . Use Heavy-Light Decomposition if the tree is extremely skewed to optimize subtree queries.

Hints: Think about the state: what information do you need to pass up from a child to a parent? If $K=0$, it's a standard tree DP. How does "any descendant" change the transition?

Q8: Count of Subarrays where the Maximum Element occurs Exactly K Times

Difficulty: Medium/Hard

Tags: Monotonic Stack, Two Pointers, Math, Combinatorics

Problem Statement: Given an array of N integers and an integer K , return the total number of continuous subarrays where the maximum element of that subarray appears exactly K times within that subarray.

Expected Approach: For every possible subarray (i, j) , find the maximum element and count its occurrences. This is $O(N^2)$ or $O(N^3)$.

Optimized Approach: Use a monotonic stack to find the left and right boundaries (L_i, R_i) for each element $A[i]$ where $A[i]$ is the unique maximum. For elements with duplicate values, define a strict ordering (e.g., the leftmost instance is the "max"). For each value, store its indices in a sorted list. For a specific index i being the m -th occurrence of value V , the K occurrences must include indices from $\text{index}[m-K+1]$ to $\text{index}[m]$. The valid range for the subarray start/end is bounded by $\text{index}[m-K]$, $\text{index}[m+1]$, and the nearest larger elements found via the monotonic stack.

Time Complexity: $O(N)$

Space Complexity: $O(N)$

Optimizations: Use a single pass with a monotonic stack to precompute the "Next Greater Element" and "Previous Greater Element" to handle the boundaries of where an element remains the maximum.

Hints: Focus on each unique value in the array. If a value X appears at indices $p_1, p_2, \dots, p_m$, how many subarrays containing exactly K of these indices have X as their maximum? Use the Monotonic Stack to find the range (L, R) where X is the king.

Q9: Pathfinding in a Grid with Limited Battery and Recharge Stations

Difficulty: Hard

Tags: Graph, Dijkstra, State-Space Search

Problem Statement: You are given an $N \times M$ grid. Each cell (r, c) is either 0 (empty), 1 (wall), or 2 (recharge station). You start at $(0, 0)$ with a battery capacity B . Moving to an adjacent cell (up, down, left, right) costs 1 unit of battery. You cannot enter a cell if your battery is 0. If you enter a cell with a recharge station (2), your battery is instantly refilled to B . Find the minimum number of steps to reach $(N-1, M-1)$.

Expected Approach: Use BFS to find the shortest path, but only tracking (r, c) leads to incorrect results because battery levels vary.

Optimized Approach: Use Dijkstra's algorithm where the state is defined as $(r, c, \text{current_battery})$. Use a 3D array `dist[N][M][B+1]` to keep track of the minimum steps to reach cell (r, c) with a specific battery level. Initialize all values to infinity. Push $(0, 0, B)$ into a priority queue. For each

neighbor, if the move is valid (battery > 0), calculate the new battery (either $\text{current_battery} - 1$ or B if the neighbor is a recharge station). Update the distance and push to the queue if the new steps are fewer than the recorded value for that state.

Time Complexity: $O(N \times M \times B \log(N \times M \times B))$

Space Complexity: $O(N \times M \times B)$

Optimizations: If all edge weights are 1 (steps), a standard BFS with the same 3D state $(r, c, \text{battery})$ can be used instead of Dijkstra to achieve $O(N \times M \times B)$ time.

Hints: Think about how the state changes when you hit a recharge station. Is it possible to reach the same cell with different battery levels? Which one is better?

Q10: Maximum Throughput in a Versioned Dependency Graph

Difficulty: Hard

Tags: Graph, Dynamic Programming, Topological Sort

Problem Statement: You have a set of N tasks with directed dependencies (a DAG). Each task i has a processing time $T[i]$ and a "version" number. You can run tasks in parallel, but there is a constraint: if task A depends on task B , A can only start after B completes. Furthermore, some tasks are "Exclusive" (marked by a flag); only one exclusive task can be running across the entire system at any given time. Find the minimum time required to complete all tasks.

Expected Approach: Use topological sort and standard critical path method (longest path in DAG) to find the completion time, ignoring the "Exclusive" constraint.

Optimized Approach: 1. Calculate the Earliest Start Time (EST) and Earliest Finish Time (EFT) for all tasks using topological sort, ignoring the Exclusive constraint. 2. Identify all tasks marked "Exclusive". 3. The bottleneck is either the longest path in the DAG or the sum of all processing times $T[i]$ of all "Exclusive" tasks. 4. However, some exclusive tasks might naturally fall into a sequence due to existing dependencies. Let L be the longest path in the DAG. Let S be the sum of durations of all Exclusive tasks. The answer is not simply $\max(L, S)$ because an exclusive task might be on the critical path. Use DP: The total time is the maximum of the critical path length and the total work required by the serial resource.

Time Complexity: $O(V + E)$ where V is the number of tasks and E is the number of dependencies.

Space Complexity: $O(V + E)$

Optimizations: Use Kahn's algorithm for topological sort to handle the dependency layering efficiently.

Hints: Treat the "Exclusive" resource as a single-core CPU. How does the total time change if the serial work exceeds the parallel critical path?
